



UCLINUX PORTING HOWTO

Monday 26th of July 2004, 04:30:56 pm. Posted in: **Embedded**. 8804 words.

Board & processor configuration:

uclinux/linux-2.4.x/arch/armnommu/config.in

In the "System" menu, add the board to be supported in the "ARM system type" selection. In this case an option ACME is added and CONFIG_ARCH_ACME defined.

```
#-----  
# S y s t e m  
#-----  
mainmenu_option next_comment  
comment 'System Type'  
choice 'ARM system type' \  
    "TI-DSC21          CONFIG_ARCH_DSC21 \  
    TI-C5471          CONFIG_ARCH_C5471 \  
    Conexant          CONFIG_ARCH_CNXT \  
    NET+ARM           CONFIG_ARCH_NETARM \  
    Triscend-A7S      CONFIG_ARCH_TA7S \  
    Triscend-A7V      CONFIG_ARCH_TA7V \  
    SWARM             CONFIG_ARCH_SWARM \  
    Samsung           CONFIG_ARCH_SAMSUNG \  
    ACME              CONFIG_ARCH_ACME \  
    Atmel             CONFIG_ARCH_ATMEL" ACME
```

Next, some of the defaults for the processor and board are defined. These options will have to be selected based on the processor characteristics. Some of the other processor configurations in the file can be used as examples.

```
if [ "$CONFIG_ARCH_ACME" = "y" ]; then  
    define_bool CONFIG_NO_PGT_CACHE y  
    define_bool CONFIG_CPU_32v4 y
```

```

define_bool CONFIG_CPU_32 y

define_bool CONFIG_CPU_26 n

define_bool CONFIG_CPU_ARM710 y

define_bool CONFIG_CPU_WITH_CACHE y

define_bool CONFIG_CPU_WITH_MCR_INSTRUCTION n

define_int CONFIG_ARM_CLK 60000000

define_int CONFIG_FORCE_MAX_ZONEORDER 5

define_bool CONFIG_SERIAL_44B0 y

define_bool CONFIG_VT y

if [ "$CONFIG_SET_MEM_PARAM" = "n" ]; then

    define_hex DRAM_BASE 0x0C000000

    define_hex DRAM_SIZE 0x00800000

    define_hex FLASH_MEM_BASE 0x00000000

    define_hex FLASH_SIZE 0x00400000

fi

```

Here, the processor clock is defined at 60Mhz, and the RAM and ROM (flash) configuration is set for this particular board. Additional options such as CONFIG_SERIAL_44B0 force the inclusion of corresponding device drivers in the linux build process.

uclinux/linux-2.4.x/arch/armnommu/Makefile

In the architecture specific Makefile, two variables need to be set. The variable TEXTADDR refers to the target location in DRAM where the final binary will be loaded and executed. Linking by the compiler is done with reference to this location. This is usually at the top of DRAM at a 32-bit aligned address. In this case, the binary is loaded at offset 0x8000 into the DRAM base address 0x0c000000. The first 32k bytes of DRAM are used to store interrupt vectors, and to pass bootup parameters to uClinux from the bootloader.

The variable MACHINE defines the directories where the processor specific files will be created, in this case: uclinux/linux-2.4.x/arch/armnommu/mach-s3c44b0x/ and uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/

```

ifeq ($(CONFIG_ARCH_ACME),y)

    TEXTADDR = 0x0c008000

    MACHINE = s3c44b0x

endif

```

uclinux/linux-2.4.x/arch/armnommu/tools/mach-types

This file holds a database of machine type macros and their numbers. A new machine requires a unique number, which is allotted by the ARM maintainers. To register, follow the steps detailed in [uclinux/linux-2.4.x/Documentation/arm/README](#). Temporarily, add a new line as shown. Select the same name (ARCH_ACME) as was used in config.in to describe the machine configuration. A unique number will also have to be selected.

#	machine_is_xxx	CONFIG_xxxx	MACH_TYPE_xxx	number
acme	ARCH_ACME	ACME	350	

Generate the header file using the script in the tools directory by executing "awk -f gen-mach-types mach-types > mach-types.h". Finally, move mach-types.h to uclinux/linux-2.4.x/include/asm-armnommu/mach-types.h

uclinux/linux-2.4.x/arch/armnommu/vmlinux-armv.lds.in

The linker script for the 32-bit processor is vmlinux-armv.lds.in. An important section to consider here is the ROM filesystem storage location. The resultant binary after the uClinux build process is a combination of the uClinux kernel and a filesystem containing selected utilities. Typically the filesystem is a ROMFS and is loaded into DRAM. There are two ways to include the filesystem image in the uClinux binary, either concatenated with the uClinux kernel, or included in the kernel itself. The former case requires movement of the filesystem into the appropriate location in DRAM, while the latter requires no such code during startup. The following code is added at the end of the .text section:

```

.text : {
    /* Real text segment */

    ...

    ...

    ...

```

```
*(.got)                /* Global offset table */

romfs_data = .;

romfs.o

romfs_data_end = .;


_etext = .;            /* End of text section */
}
```

The romfs image romfs.o is written at the location romfs_data in the final binary. This variable is referenced in the blkmem driver.